

Algorithmique 1

3. Récursivité

Johan Thapper
2023/2024



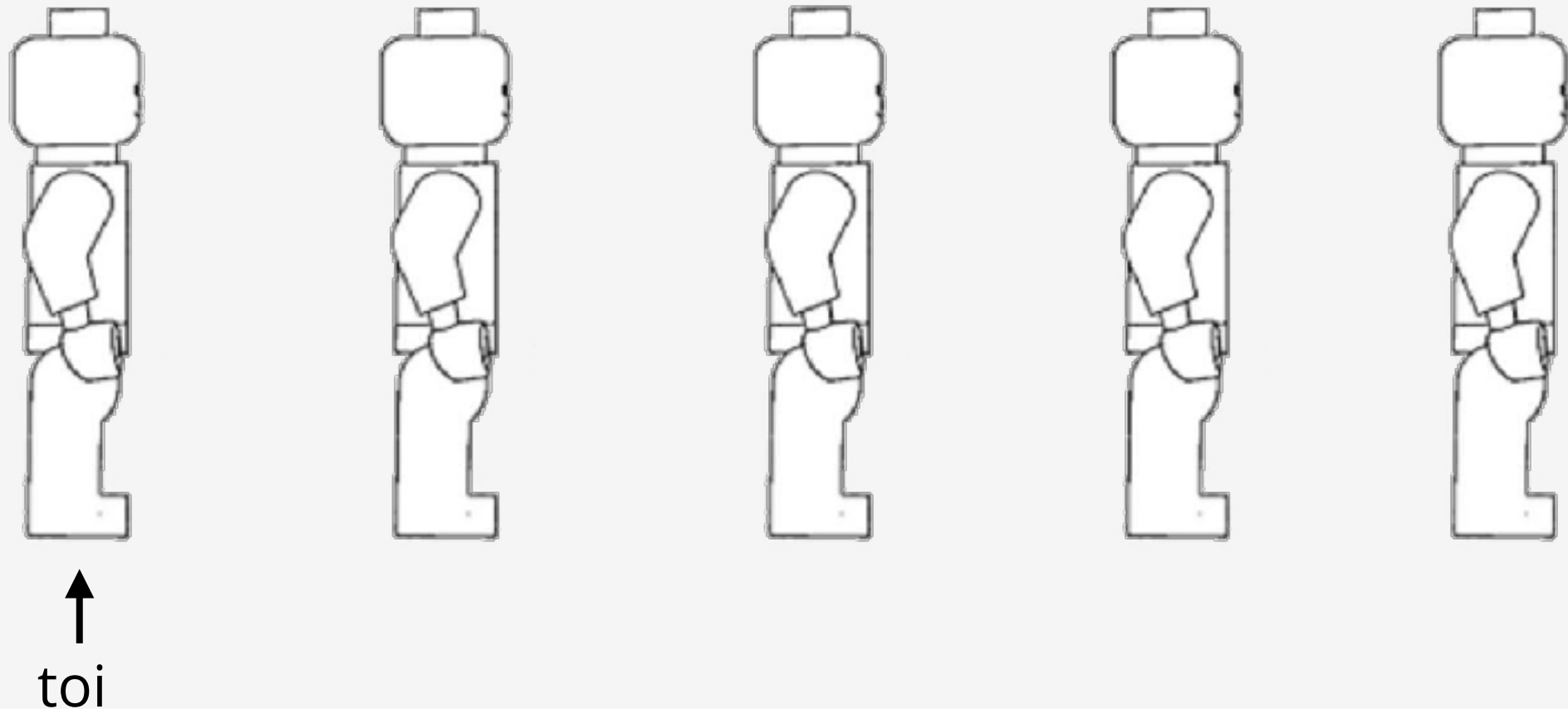
La récursivité

- La récursivité est la définition d'une notion en termes d'elle même.



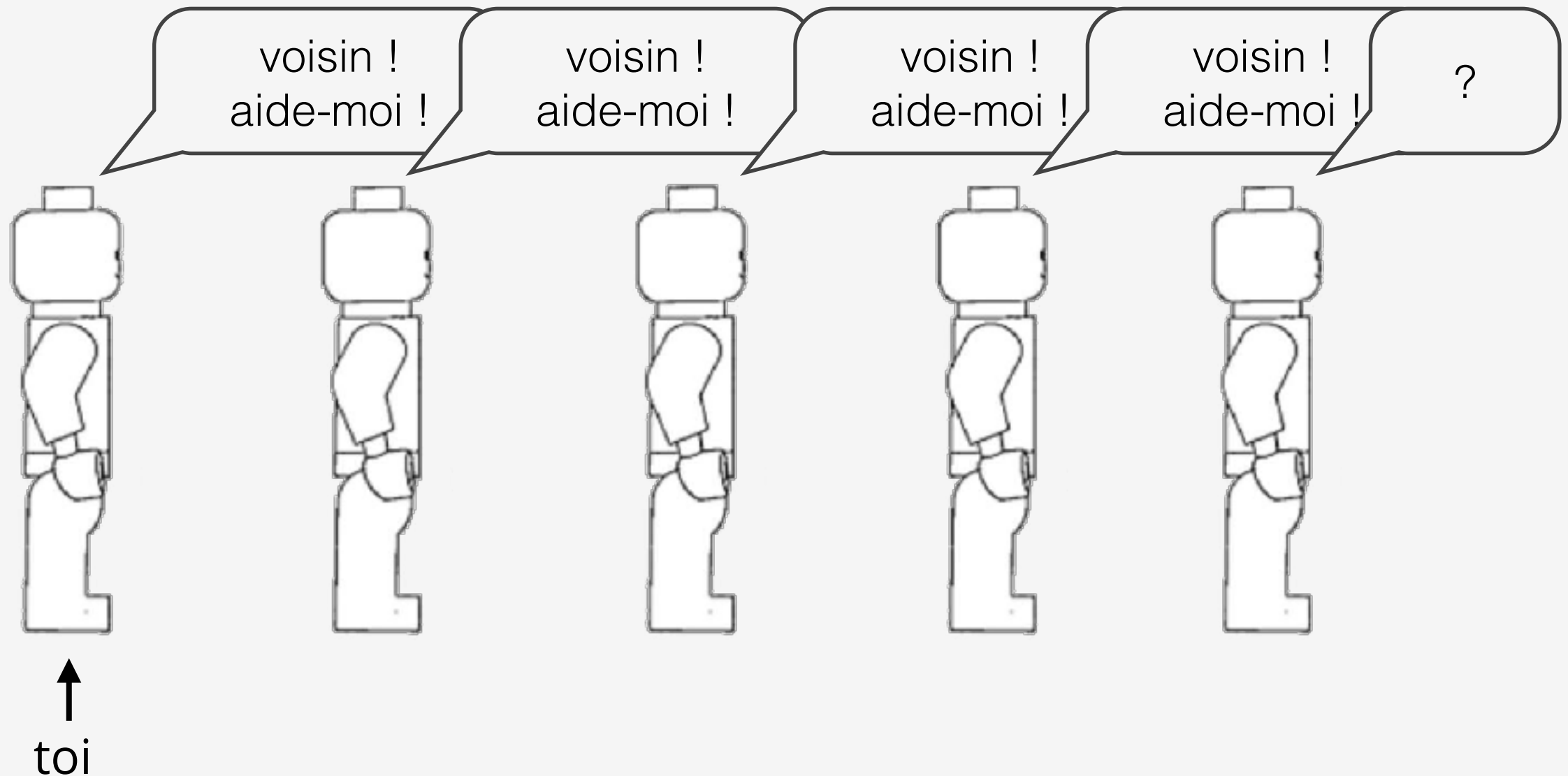
Dans la file d'attente

- ▶ Tu veux savoir combien de personnes sont devant toi
- ▶ Tu ne peux voir que ton voisin, donc tu ne peux pas les compter
- ▶ Mais tu peux lui poser une question
- ▶ Comment résoudre ce problème avec la récursivité ?



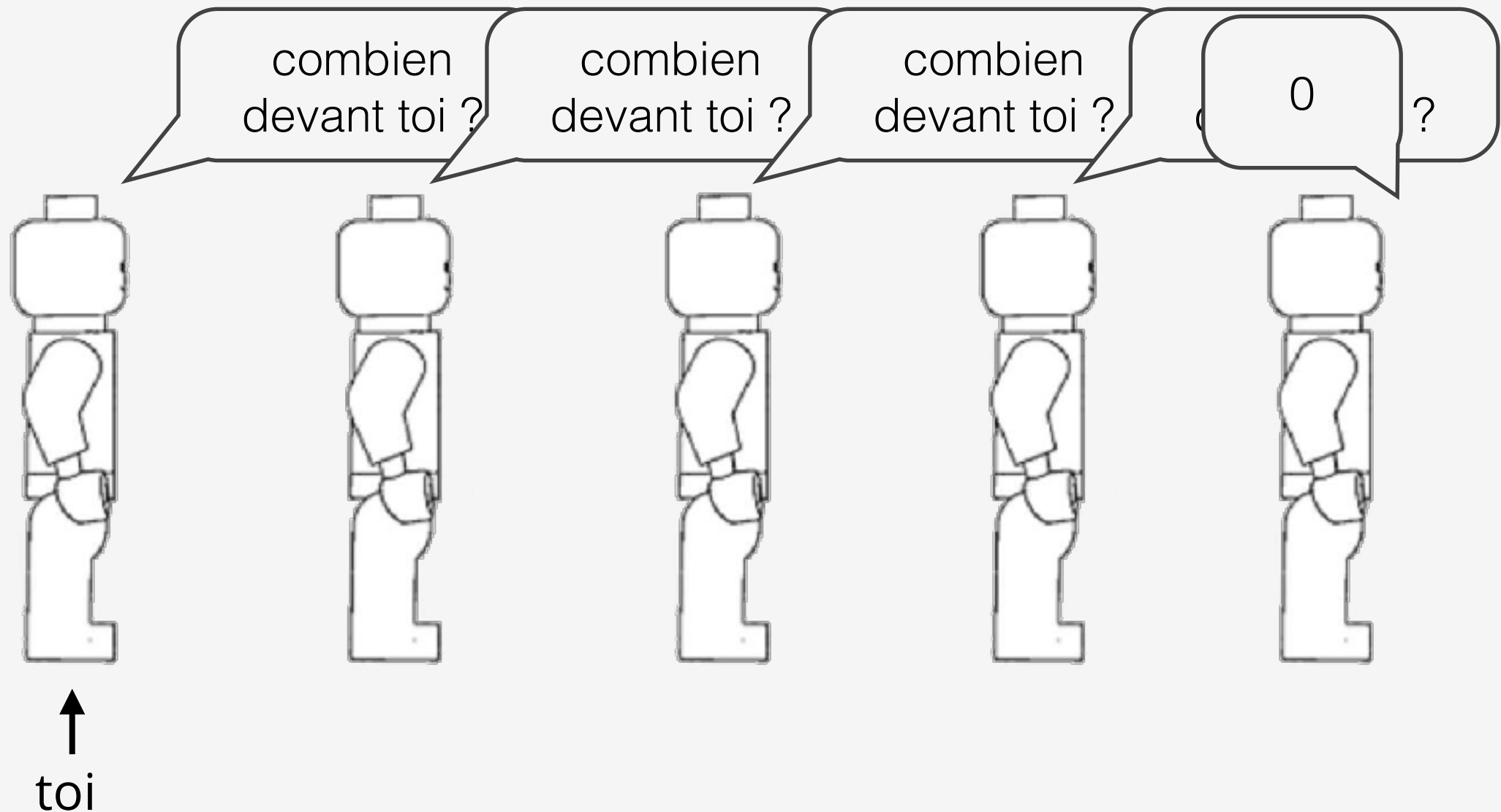
Dans la file d'attente

- ▶ Que peut être une bonne question ?
- ▶ Quelles informations de ton voisin pourraient t'aider ?



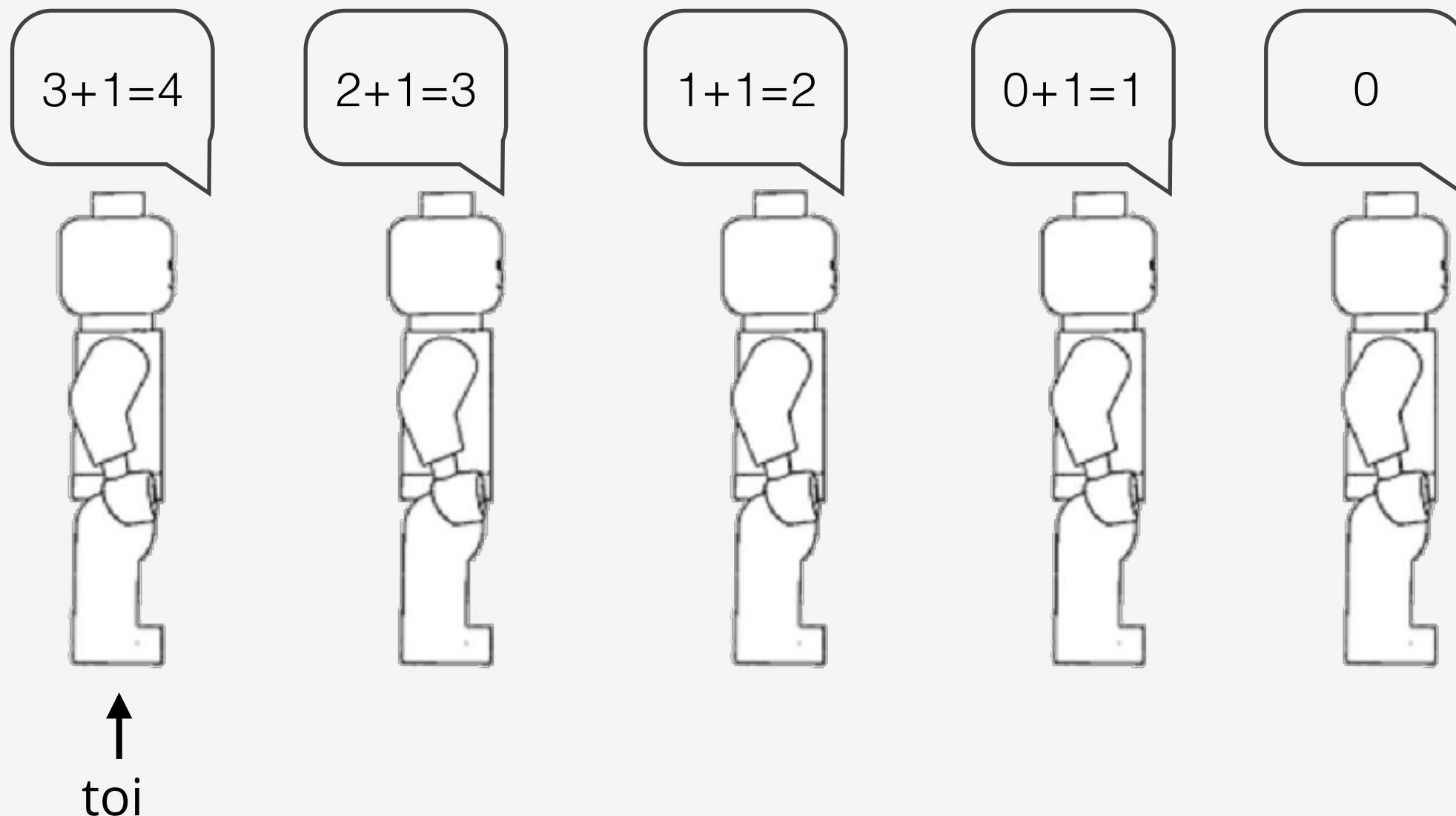
Dans la file d'attente

- ▶ Pour savoir combien de personnes sont devant toi
 - ❖ s'il n'y a personne devant toi, alors répondre 0
 - ❖ sinon, il y a quelqu'un devant toi, demande-lui combien sont devant lui



Dans la file d'attente

- ▶ Pour savoir combien de personnes sont devant toi
 - ❖ s'il n'y a personne devant toi, alors répondre 0
 - ❖ sinon, il y a quelqu'un devant toi, demande-lui combien sont devant lui
 - ❖ si sa réponse est N, alors répondre N+1



La récursivité

- ▶ Une fonction est dite **récursive** si elle contient un appel à elle-même.
- ▶ En programmation, la récursivité est souvent utilisée pour effectuer une **répétition**, en remplaçant une boucle.
- ▶ *Fonction récursive — fonction itérative*
 - ❖ On fait parfois contraster une fonction récursive par une **fonction itérative** (une fonction écrite avec des boucles for/while pour effectuer une répétition) pour une même tâche.

La récursivité

- ▶ Certains algorithmes et fonctions sont plus naturellement écrits en récursive. Par exemple, si on a déjà une définition par récurrence
- ▶ Exemple : la fonction *factorielle* $n!$ est définie par
 - ❖ $0! = 1$
 - ❖ $n! = n \times (n-1)!$ si $n > 0$
- ▶ En C, la version récursive reflète la structure de cette définition. Quelle version est récursive/itérative ci-dessous ?

(a)

```
int factorial(int n) {  
    int res = 1;  
    while (n > 0) {  
        res = res * n;  
        n--;  
    }  
    return res;  
}
```

(b)

```
int factorial(int n) {  
    if (n == 0)  
        return 1;  
    else  
        return n*factorial(n-1);  
}
```

Principe d'un algorithme récursif

- ▶ Tout algorithme récursif comprend au moins 2 cas
- ▶ **Une ou plusieurs conditions d'arrêt**
 - ❖ pour traiter une instance simple du problème à laquelle on peut immédiatement donner une réponse
 - ❖ dans l'exemple : réponds "0" s'il n'y a personne devant toi
- ▶ **Un ou plusieurs appels récursifs**
 - ❖ pour traiter une instance plus complexe à laquelle on n'a pas immédiatement une réponse, mais qui peut être réduit à une instance plus simple du même problème
 - ❖ dans l'exemple : demande à ton voisin et réponds "N+1"

Décomposition récursive

- ▶ On appelle **décomposition récursive** la façon de découper la résolution d'un problème en conditions d'arrêt et appels récursifs
- ▶ Pour en trouver une, répondre aux questions suivantes :
 - ❖ pour quelles instances la réponse est-elle évidente ?
(conditions d'arrêt)
 - ❖ pour un tableau : souvent un *tableau vide* (de longueur 0)
 - ❖ qu'est-ce qu'est une instance *plus simple* du problème ?
 - ❖ pour un tableau : souvent un tableau *plus court*
 - ❖ supposer que l'algorithme sait déjà répondre aux instances plus simples (par les appels récursifs)
 - ❖ comment pouvez-vous répondre au problème initial en connaissant les réponses aux instances plus simples ?

Partie effective

- ▶ Dans les exemples qui suivent (ainsi qu'au td et au tp) on considère une partie d'un tableau compris entre deux indices **lo** et **hi** (inclus).
- ▶ On appelle **la partie effective** d'un tableau les éléments du tableau entre les indices **lo** et **hi**. Si $lo > hi$, alors la partie effective est vide.
- ▶ les arguments **lo** et **hi** à une fonction remplacent **size**.
- ▶ Cela rend l'écriture des fonctions en récursive plus naturelle.

<i>0</i>	<i>1</i>	<i>2</i>	<i>3</i>	<i>lo</i> <i>4</i>	<i>5</i>	<i>6</i>	<i>7</i>	<i>hi</i> <i>8</i>	<i>9</i>
5	3	7	2	9	13	8	11	1	2

Quelle est la somme des éléments de la partie effective du tableau ci-dessus ?

Somme d'une partie d'un tableau

- Objectif : trouver la somme des éléments de la partie effective d'un tableau

```
int sum_iter(int t[], int lo, int hi) {  
    int sum = 0;  
    int i;  
    for (i = lo; i <= hi; i++)  
        sum = sum + t[i];  
    return sum;  
}
```

*Exercice : trouver une décomposition
récursive de cette fonction*

Somme d'une partie d'un tableau

- ▶ Condition d'arrêt (si $lo > hi$)
 - ❖ la partie effective du tableau est vide
 - ❖ renvoyer 0 (la somme de rien)
- ▶ Appel récursif (sinon)
 - ❖ laisser la fonction chercher elle-même la réponse pour *la suite* $t[lo+1]+t[lo+2]+\dots+t[hi]$ (la longueur est plus petite — plus simple !)
 - ❖ y ajouter $t[lo]$ et renvoyer la réponse

```
int sum(int t[], int lo, int hi) {  
    if (lo > hi)  
        return 0;  
    else  
        return t[lo] + sum(t, lo+1, hi);  
}
```

Recherche linéaire

- Objectif : chercher un élément dans la partie effective d'un tableau d'éléments

```
int find_iter(int t[], int lo, int hi, int elt) {  
    int i;  
    for (i = lo; i <= hi; i++)  
        if (t[i] == elt)  
            return 1  
    return 0;  
}
```

*Exercice : trouver une décomposition
récursive de cette fonction*

Recherche linéaire

- ▶ Condition d'arrêt (si $lo > hi$)
 - ❖ renvoyer 0
- ▶ Condition d'arrêt (si $t[lo] == elt$)
 - ❖ renvoyer 1
- ▶ Appel récursif (sinon)
 - ❖ laisser la fonction chercher elle-même parmi les éléments $t[lo+1], t[lo+2], \dots, t[hi]$ (la longueur est plus petite — plus simple !)
 - ❖ renvoyer la réponse obtenue

```
int find(int t[], int lo, int hi, int elt) {  
    if (lo > hi)  
        return 0;  
    else if (t[lo] == elt)  
        return 1;  
    else  
        return find(t, lo+1, hi, elt);  
}
```

La première règle d'or de la récursivité

- ▶ La première condition d'arrêt est identique pour les deux fonctions **sum** et **find**

La première règle

La première condition doit être une condition d'arrêt.
Pour une fonction qui travaille sur un tableau, utiliser
"est-il vide?" comme la première condition.

Doublons

- ▶ Objectif : détecter s'il y a des **doublons** dans la partie effective d'un tableau d'éléments

*Exercice : écrire une version récursive de cette fonction (sans utiliser de boucles)
Vous pouvez réutiliser la fonction find de l'exemple précédent*

Doublons

- ▶ Condition d'arrêt (si $lo > hi$)
 - ❖ renvoyer 0
- ▶ Appel "récuratif" (si $t[lo]$ est dans la partie $t[lo+1], t[lo+2], \dots, t[hi]$)
 - ❖ renvoyer 1
- ▶ Appel récursif (sinon)
 - ❖ $t[lo]$ n'apparaît pas dans la partie $t[lo+1], t[lo+2], \dots, t[hi]$
 - ❖ chercher des doublons dans la partie $t[lo+1], t[lo+2], \dots, t[hi]$
 - ❖ renvoyer la réponse obtenue

```
int duplicates(int t[], int lo, int hi) {  
    if (lo > hi)  
        return 0;  
    else if (find(t, lo+1, hi, t[lo]))  
        return 1;  
    else  
        return duplicates(t, lo+1, hi);  
}
```

Mystère

- ▶ Que renvoie l'appel `mystere(10)` ?

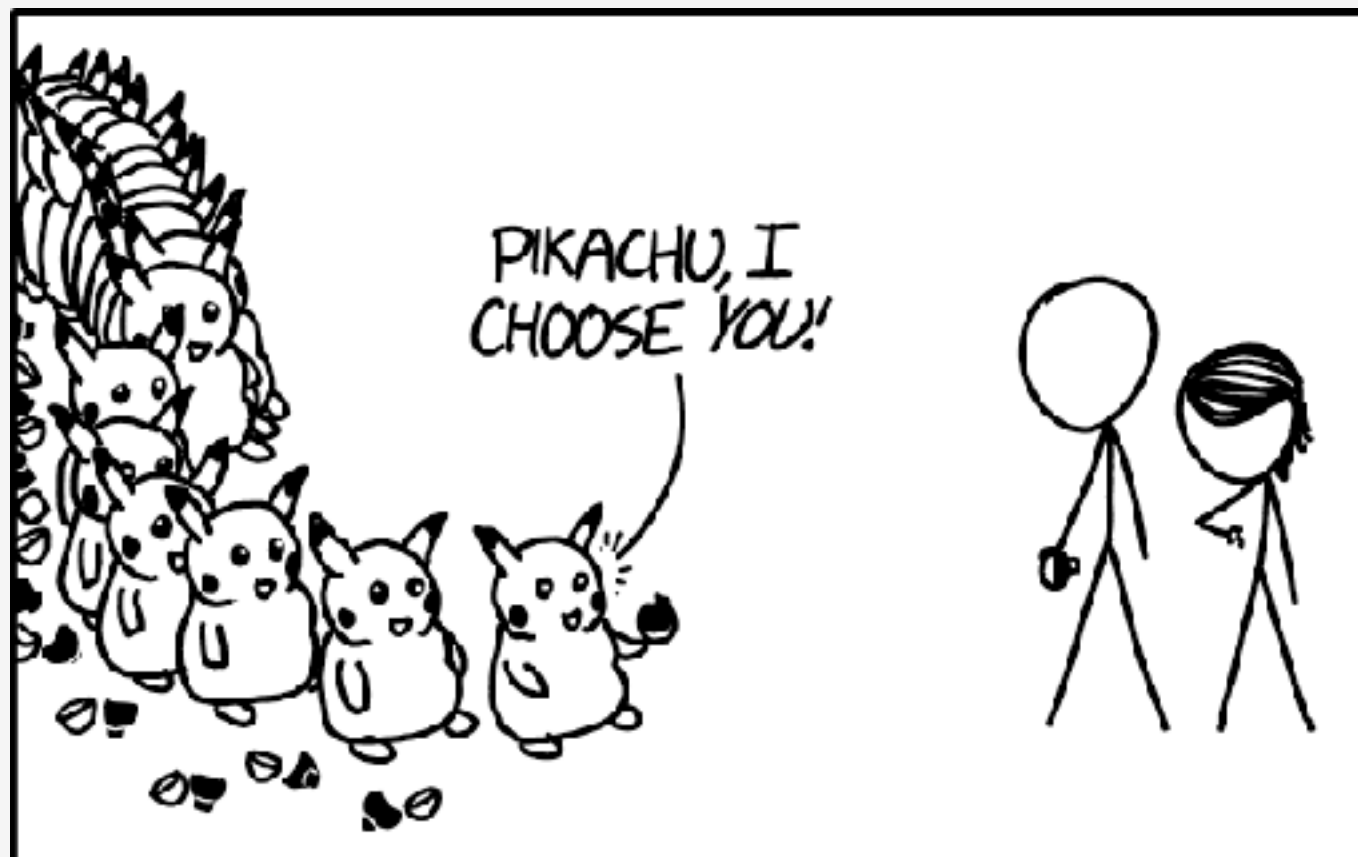
```
int mystere(int n) {  
    if (n == 0)  
        return 1;  
    else  
        return 2*mystere(n/2+1);  
}
```

- ▶ Les appels récursifs se font dans l'ordre suivant
 - ❖ `mystere(10)`
 - ❖ `mystere(6)`
 - ❖ `mystere(4)`
 - ❖ `mystere(3)`
 - ❖ `mystere(2)`
 - ❖ `mystere(2)`
 - ❖ ... (boucle infinie)

La deuxième règle d'or de la récursivité

La deuxième règle

Il faut toujours changer les arguments de l'appel récursif pour s'approcher d'une condition d'arrêt.



<https://xkcd.com/1516/>

Inverser un tableau

- ▶ Objectif : inverser le contenu d'un tableau

<i>lo</i>	<i>lo+1</i>	<i>...</i>			<i>...</i>	<i>hi-1</i>	<i>hi</i>
1	5	3	9	4	2	3	7

- ▶ Condition : le tableau est-il vide ?

- ❖ si oui, alors il n'y a rien à inverser, donc c'est fini.
- ❖ sinon, on échange le contenu de $t[lo]$ et $t[hi]$:

<i>lo</i>	<i>lo+1</i>	<i>...</i>			<i>...</i>	<i>hi-1</i>	<i>hi</i>
7	5	3	9	4	2	3	1

- ❖ Ensuite on inverse *le reste* :

<i>hi</i>				<i>lo</i>			
7	3	2	4	9	3	5	1

```
void reverse(int t[], int lo, int hi) {  
    if (lo > hi)  
        return;  
    else {  
        swap(&t[lo], &t[hi]);  
        reverse(t, lo+1, hi-1);  
    }  
}
```

```
void swap(int *a, int *b) {  
    int c = *a; *a = *b; *b = c;  
}
```

Exécution

```
void reverse(int t[], int lo, int hi) {  
    if (lo > hi)  
        return;  
    else {  
        swap(&t[lo], &t[hi]);  
        reverse(t, lo+1, hi-1);  
    }  
}
```

1	2	3	4	5	6
---	---	---	---	---	---

reverse([1,2,3,4,5,6], 0, 5)

6	5	4	3	2	1
---	---	---	---	---	---

<i>lo</i>	<i>lo+1</i>			<i>hi-1</i>	<i>hi</i>
6	2	3	4	5	1

```
else {  
    swap(&t[lo], &t[hi]);  
    reverse(t, lo+1, hi-1);  
}
```

6	5	4	3	2	1
---	---	---	---	---	---

<i>lo</i>	<i>lo+1</i>	<i>hi-1</i>	<i>hi</i>		
6	5	3	4	2	1

```
else {  
    swap(&t[lo], &t[hi]);  
    reverse(t, lo+1, hi-1);  
}
```

6	5	4	3	2	1
---	---	---	---	---	---

<i>lo</i>	<i>hi</i>				
6	5	4	3	2	1

```
else {  
    swap(&t[lo], &t[hi]);  
    reverse(t, lo+1, hi-1);  
}
```

6	5	4	3	2	1
---	---	---	---	---	---

<i>hi</i>	<i>lo</i>				
6	5	4	3	2	1

```
if (lo > hi)  
    return;
```

Simplification et wrapper

La troisième règle

Ne simplifier une fonction qu'après qu'elle soit correcte.

```
void reverse(int t[], int lo, int hi) {  
    if (lo > hi)  
        return;  
    else {  
        swap(&t[lo], &t[hi]);  
        reverse(t, lo+1, hi-1);  
    }  
}
```

```
void reverse_aux(int t[], int lo, int hi) {  
    if (lo <= hi) {  
        swap(&t[lo], &t[hi]);  
        reverse_aux(t, lo+1, hi-1);  
    }  
}
```

```
void reverse(int t[], int size) {  
    reverse_aux(t, 0, size-1);  
}
```

wrapper



une fonction wrapper sert à donner une signature différente à une autre fonction

Les trois règles d'or de la récursivité

La première règle

La première condition doit être une condition d'arrêt.
Pour une fonction qui travaille sur un tableau, utiliser
"est-il vide?" comme la première condition.

La deuxième règle

Il faut toujours changer les arguments de l'appel
récursif pour s'approcher d'une condition d'arrêt.

La troisième règle

Ne simplifier une fonction qu'après qu'elle soit
correcte.

Pourquoi apprendre la récursivité ?

- ▶ Une façon différente de résoudre des problèmes
- ▶ Mieux adapté à certains problèmes
- ▶ Amène à un code élégant, simple et court (si bien utilisé)
- ▶ L'informatique est pleine de notions récursives
 - ❖ systèmes de fichiers
 - ❖ autres structures arborescentes (voir Algorithmique 2)
 - ❖ expressions régulières
- ▶ Il y a des langages de programmation (**langages fonctionnels**) qui sont entièrement (ou au moins fortement) basés sur la récursivité
 - ❖ LISP, Scheme, ML, Haskell, Scala
- ▶ On va rencontrer la récursivité plusieurs fois au long du cours